Foundations in Data Engineering

Performance and Efficiency

Jana Giceva



Recap from previous lecture

- Simple command-line tools can ***reveal key structure*** in ***raw data***.
- Small ***composable*** transformations scale also to ***larger systems.***
- Understanding the ***basics*** builds ***intuition for modern data engineering tools.***
- Now we want to ***learn***:
  - When should we use a more ***powerful tool***? When should we ***scale-up/out***?

Understanding the Performance Spectrum

- **Now we want to understand the performance spectrum.**
  - What kind of performance can we expect from a *single box*?
  - How large is the spread from *high-level tools* to *hand-written code*?
  - What are the *theoretical limits*?
  - What is the *impact* of *careful programming*?
  - How large would a *cluster* have to be to *improve performance*?
  - Ultimately, we want to know *which tools* and *hardware* we need for a *specific problem*.

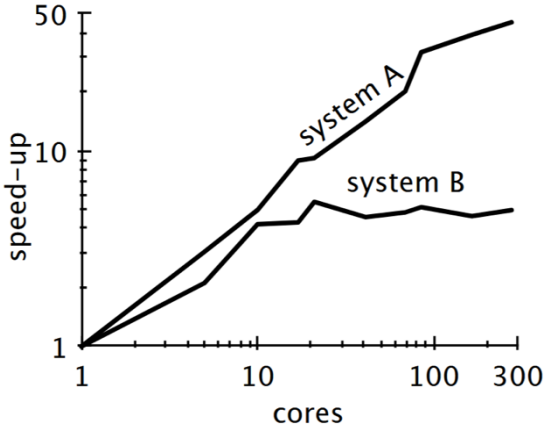
- To **understand that, we:**
  - First try to understand the current status of modern hardware
  - Take a simple problem, solve it with different programs, profile it and optimize as much as needed.

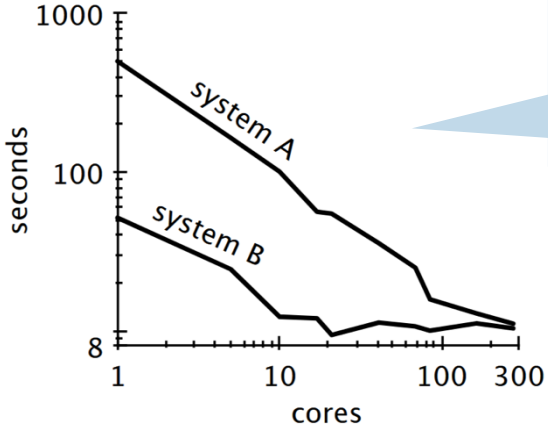
The importance of getting an efficient baseline

- *"You can have a second computer once you've shown you know how to use the first one"*
  – Paul Barham

- Scalability! But at what COST? – HotOS'15 paper by Frank McSherry, Michael Isard, and Derek Murray
  - many BigData systems are evaluated in terms of their ***scalability***, against no proper ***baselines.***

- How can that lead to misleading observations / conclusions?





Despite being less *scalable*
`system B` consistently outperforms
`system A` across various core counts.

COST – more details

- Scaling to more cores and machines is absolutely the right thing to do *when needed* – but we need to be careful with our implementation.

scalable system	cores	twitter	uk-2007-05
GraphChi [12]	2	3160s	6972s
Stratosphere [8]	16	2250s	-
X-Stream [21]	16	1488s	-
Spark [10]	128	857s	1759s
Giraph [10]	128	596s	1235s
GraphLab [10]	128	249s	833s
GraphX [10]	128	419s	462s
Single thread (SSD)	1	300s	651s
Single thread (RAM)	1	275s	-

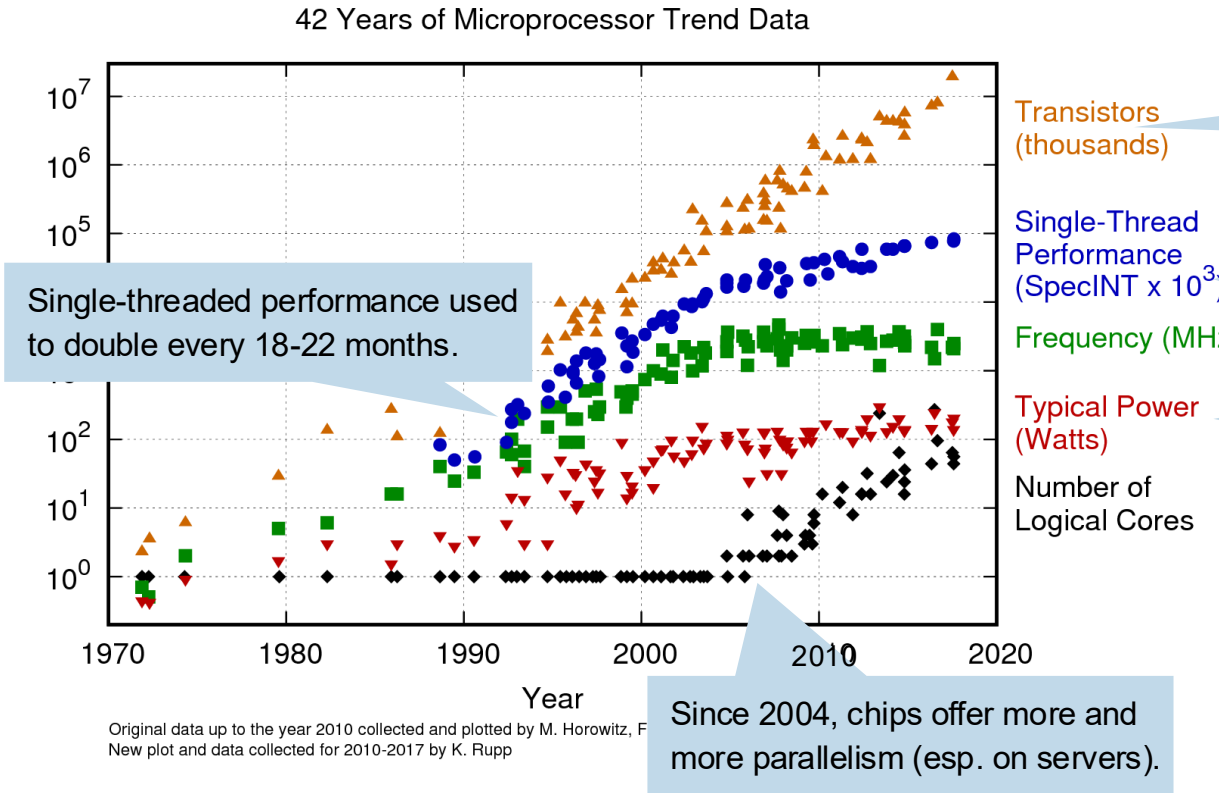
20 iterations of PageRank on the Twitter dataset.

A careful implementation on a single thread can outperform clusters using ,machines with up to 128 cores.

If the twitter dataset would fit on an SSD or DRAM, should we even scale beyond one machine?

Hardware trends

Hardware trends



Single-threaded performance used to double every 18-22 months.

Slowing down of Moore's law.

Now, single-threaded performance is stagnating!

End of Dennard scaling in 2004!

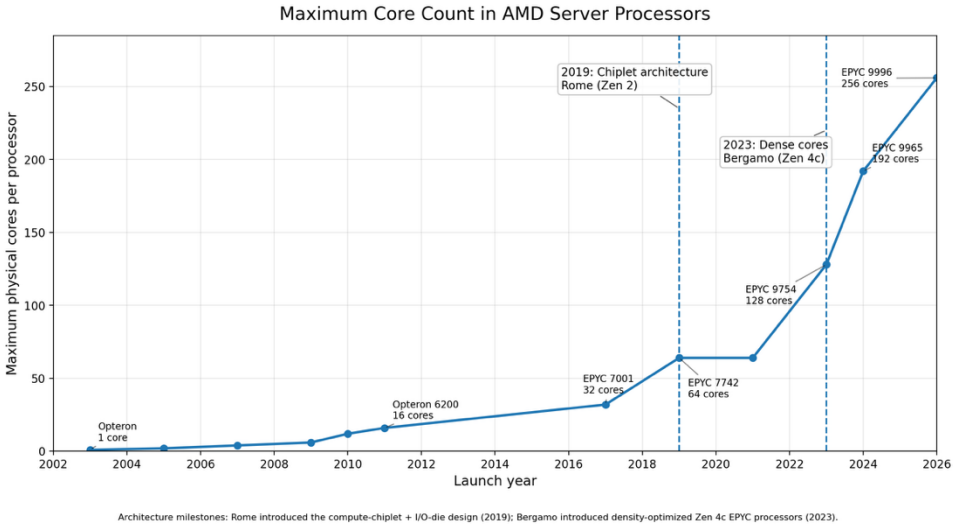
Since 2004, chips offer more and more parallelism (esp. on servers).

Effect 1: increasing parallelism

- Since the ***end of Dennard scaling*** in 2004, historic switch for the microprocessor industry to ***increase core count*** instead of single processor's efficiency

- Go from relying solely on ***instruction level parallelism*** (ILP) to ***data-level parallelism*** (DLP) and ***thread-level parallelism*** (TLP).

More on this next lecture



Major implications for software:

- Previously hardware and compiler could ***implicitly*** exploit ILP without the programmer's attention and help.

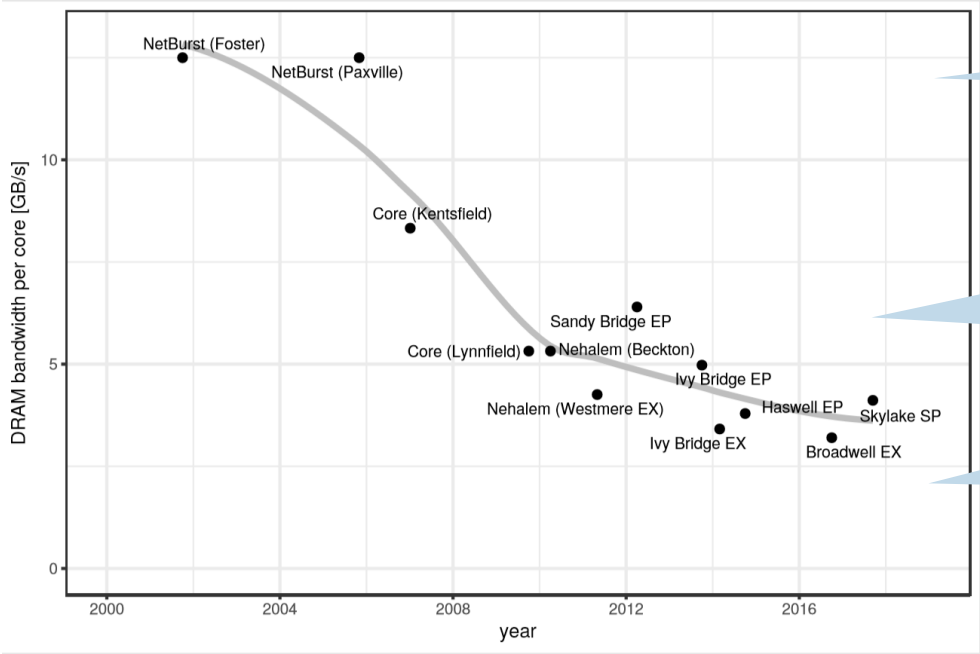
- TLP and DLP, however, are ***explicitly*** parallel and require restructuring of the application so that it can exploit the parallelism

Effect 1: increasing parallelism

- In addition to **core count**, which exploits **thread-level parallelism** (TLP)
- The issue width of **vectorized instructions** has significantly increased to exploit **data-level parallelism (DLP)** within general purpose processors
- **Intel's SIMD (Single Instruction Multiple Data) evolution** summarized:
  - 1997: MMX 64-bit (Pentium 1),
  - 1999: SSE 128-bit (Pentium 3),
  - 2011: AVX 256-bit `float` (Sandy Bridge)
  - 2013: AVX2 256-bit `int` (Haswell),
  - 2017: AVX-512 512-bit (Skylake)
  - Announced in 2024: AVX 10.2 512-bit, new data movement, transforms, and type instructions
- And the increase of popularity and use of Vector and GPU architectures

Effect 2: hitting the Memory wall

- The memory bandwidth does not increase as fast as the core count



Memory has become the new disk

Need for multi-tier cache hierarchy, and

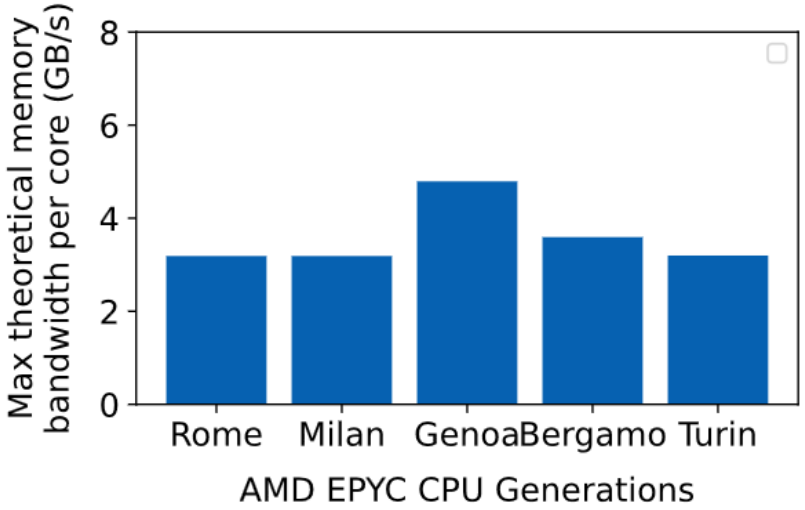
careful use of registers and the cache hierarchy

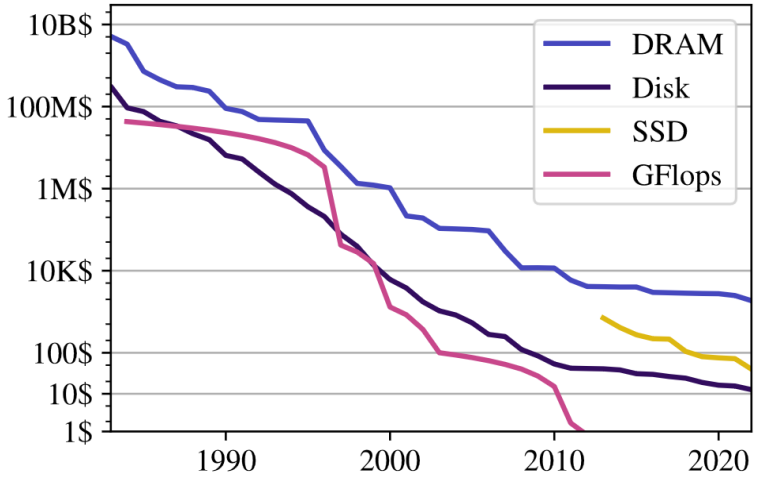
But, caches bring their own benefits and pitfalls:
Pollution, MESI protocol, timing attacks

img src: http://db.in.tum.de/teaching/ws1718/dataprocessing/chapter1.pdf by Dr. Viktor Leis

Effect 2: hitting the Memory wall

- The memory bandwidth does not increase as fast as the core count





Cost of compute (cost/Gflops), memory and storage (cost/TB) over time

Results from “Chronis et al. Databases in the Era of Memory-Centric Computing” *CIDR 2025*

Effect 3: towards Hardware Specialization

Dennard law: *as transistors get smaller, their power density stays constant, so that power use stays in proportion with area.*

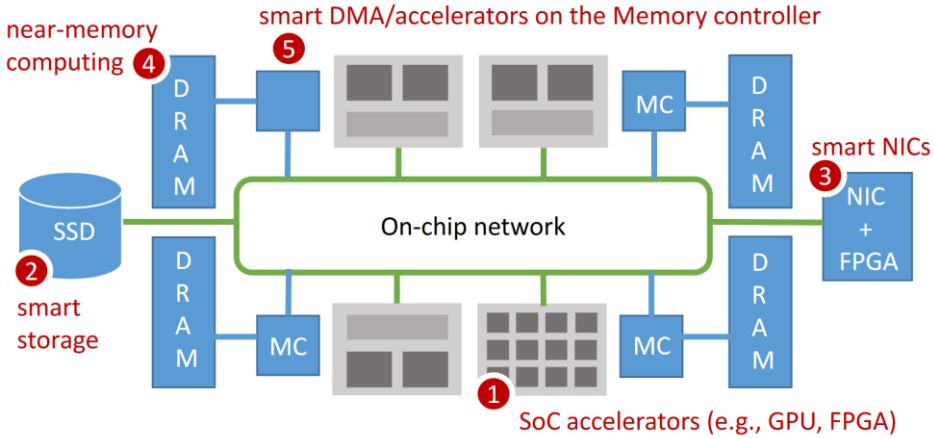
→ **Break down** in the mid-2020s when voltage scaling Could no longer keep pace with transistor size reductions – power wall in modern processors.

Domain specific architectures (DSAs)*

- GPUs, TPUs, NPUs, FPGAs, ASICs, etc.

Active hardware:

- smart storage, smart NICs, programmable switches, processing-in-memory, smart DMA engines, programmable memory controllers?

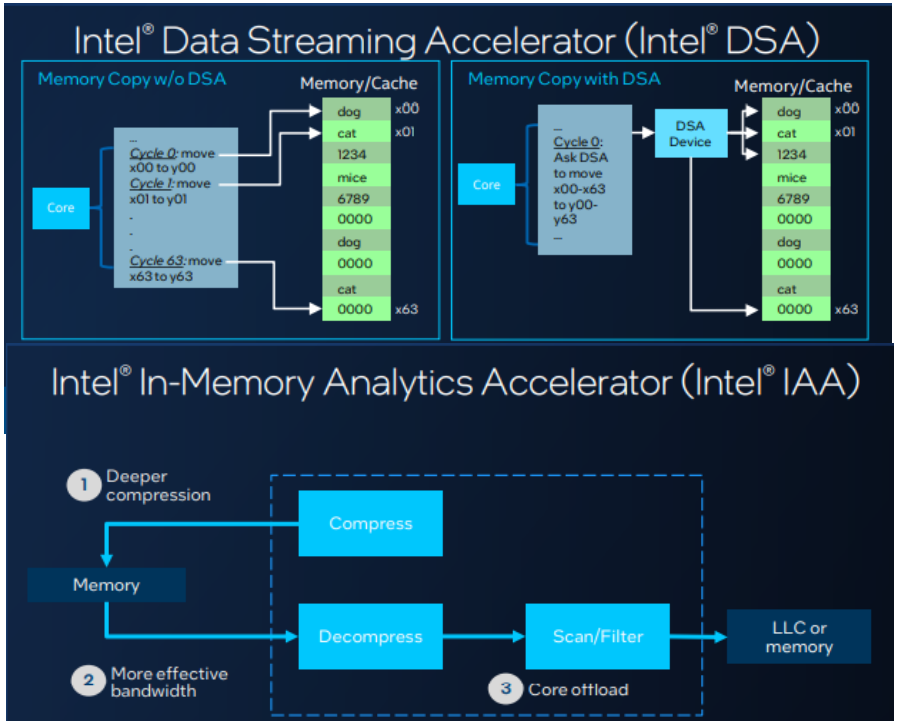


* https://cacm.acm.org/magazines/2019/2/234352-a-new-golden-age-for-computer-architecture/fulltext

Example hardware specialization today

- **Historical: Only specialized HW before rise of general purpose**
  - Gamma database machine, Lisp machine

- **Today on the market:**
  - Special instructions: AVX-512, AMX
  - Graphics Processing Units (GPUs)
  - Dedicated AI accelerators (TPUs, Trainium, etc)
  - NPUs in client CPUs (Apple Neural engine, etc.)
  - DPUs / IPUs/ SmartNICs (NVIDIA Bluefield, AWS Nitro, AMD Pensando).
  - Computational storage / Smart SSDs
  - Programmable switches
  - FPGAs
  - CXL



- **Also today:** many ongoing research and industry projects

AMD EPYC Venice

AMD EPYC 9996 (6th gen) – launch July 2026

- Zen 6 Architecture (and Zen 6c chiplet design)
- with up to 256 cores per CPU, 512 threads
- With up to 4TB DRAM (two sockets) (DDR5 RDIMM / MRDIMM up to 1024 GB/s / 1638 GB/s per socket)
- Large 1024 MB L3 cache

- https://www.amd.com/en/products/processors/server/epyc/9006-series/amd-epyc-9996.html
- https://lenovopress.lenovo.com/lp2028-introduction-to-mrdimm-memory-technology

Beyond CPU trends and Accelerators

- **CXL: Compute Express Link**
  - Open standard cache-coherency interconnect between CPUs and accelerators, and memory devices
  - Three device types:
    - Type 1: accelerators without local device memory
    - Type 2: accelerators with local memory – GPUs, ASICs, FPGAs
    - Type 3: memory expansion devices
  - CXL 3.x: goes beyond towards fabric-attached memory, memory pooling and sharing

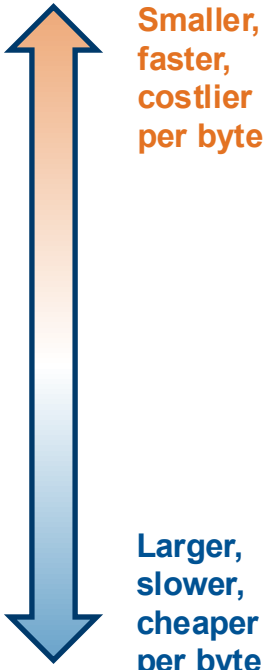
src: https://www.rambus.com/blogs/compute-express-link/

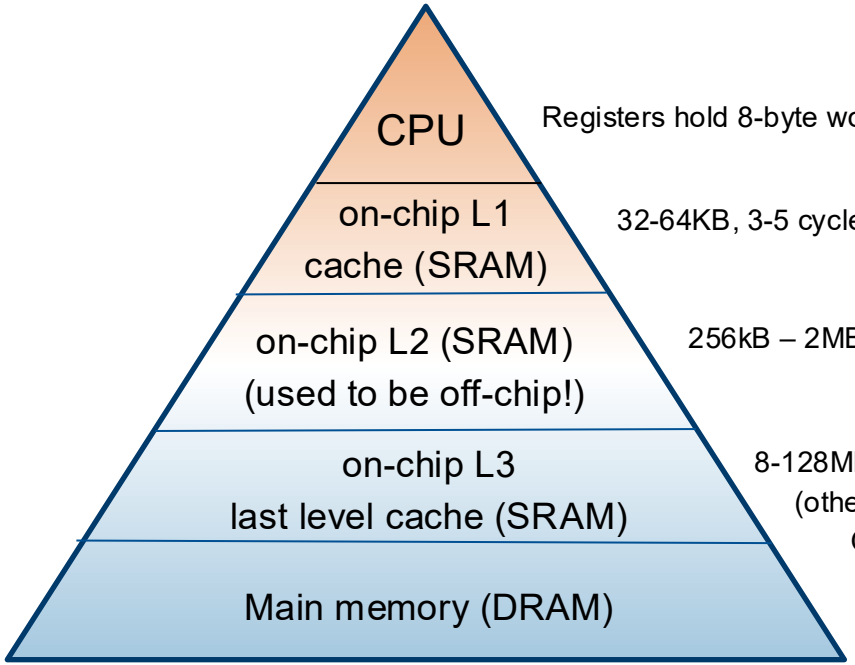
- **Low-latency high-bandwidth networks**
  - InfiniBand (IB) is a networking standard used in HPC that has very high throughput and low latencies
  - Already 400-800G Ethernet deployed in data-centers, emerging 1.8T Ethernet

Src: https://osi.rosenberger.com/news-media/400-gigabit-800g-ethernet/

Memory Hierarchy

Memory Pyramid (Caches)





Registers hold 8-byte words

32-64KB, 3-5 cycles (~1ns) , 1-2k GB/s

256kB – 2MB, ~10 -15cycles (~3-5ns), ~0.5-1k GB/s

8-128MB, ~30-50 cycles (10-30ns), ~100-500 GB/s
(other technologies HBM: up to 3-5 TB/s,
GDDR: up to 1 TB/s)

Slots*32/64GB, ~60-100ns,
#channels x 32-64 GB/s (12-16 channels)

SSD (NVMe): 0.5-4 TB, ~100us, 7-12 GB/s
HDD: 1-20TB, ~3ms, 100-250 MB/s

Latency numbers one should know

- Originally shown by Jeff Dean

Latency Numbers: 2012 vs. 2026

	Operation	2012 (Jeff Dean)	2026 (Representative)
CPU (Computation)	CPU cycle	~0.3 ns	**~0.2 ns**
	L1 cache reference	0.5 ns	**~1 ns**
	Branch mispredict	5 ns	**~3–5 ns**
	L2 cache reference	7 ns	**~3 ns**
	L3 / LLC reference	—	**~10–20 ns**
Memory	Main memory (DRAM) access	100 ns	**~80–120 ns**
	Read 1 MB sequentially from memory	250 µs	**~10–30 µs**
Storage (SSD)	SSD random read (4 KB)	150 µs	**~50–100 µs**
	Read 1 MB sequentially from SSD	1 ms	**~35–100 µs**
Storage (HDD)	Disk seek (random access)	10 ms	**~5–10 ms**
	Read 1 MB sequentially from HDD	20 ms	**~4–10 ms**
Network	Datacenter network RTT (within a datacenter / same region)	500 µs	**< 1 ms** (same AZ)
	California → Netherlands → California (intercontinental RTT)	150 ms	**~140–160 ms**

It's the memory/storage hierarchy, extended to storage / network and accounts for other key aspects such as context switches, compression, etc.

Bandwidth improved dramatically.
Latency often didn't.

- SSD seq. bandwidth: ~1 GB/s → 14 GB/s
- DRAM bandwidth: x10 depending on platform

bandwidth improve, latency not that much

src for the 2026 numbers: https://cpus.me/misc/latency/

Q: Why is this important?

- Could let us do back-of-envelope calculations on what we should expect to get.
- Can make us reason on what to avoid on the critical path / hot loop

- Let’s investigate a few examples on how optimizing the way we read / organize data can make our program run faster without changing the algorithm itself.

A motivating example (memory access)

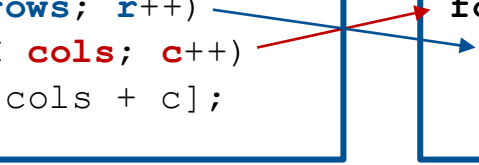
Task: sum up all entries in a two-dimensional array

Alternative 1:

```
for (r = 0; r < rows; r++)
  for (c = 0; c < cols; c++)
    sum += src[r*cols + c];
```

Alternative 2:

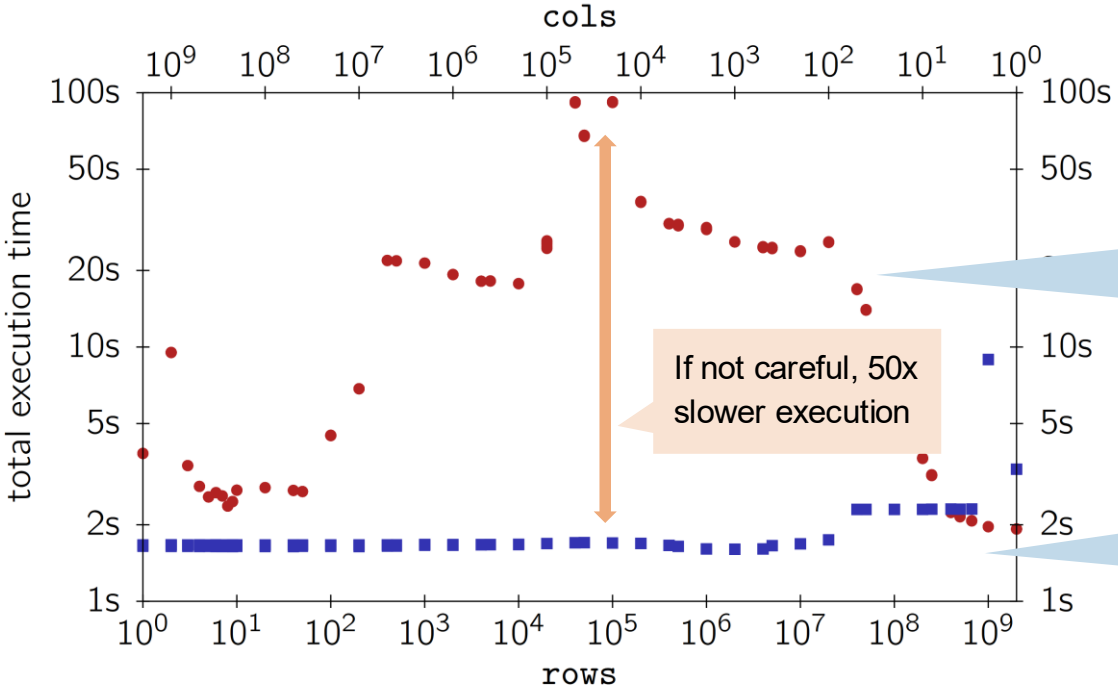
```
for (c = 0; c < cols; c++)
  for (r = 0; r < rows; r++)
    sum += src[r*cols + c];
```



Both alternatives touch the same data, but in different order.

A motivating example (memory access)

Task: sum up all entries in a two-dimensional array



Alternative 2 iterates over the elements column-wise, and its performance is highly dependent on whether the working set size fits in the memory hierarchy.

Alternative 1 iterates over the elements row-wise, which is more friendly to the underlying micro-architectural features.

So how do we approach it?

- Careful consideration on ***how we access the data***
  - Sequential is always better than random (for DRAM and HDDs)!
- Think about ***how we should layout the data*** in memory (or storage).
  - Which layout is more suitable for our workload?
  - *e.g.,* Column vs row store in databases (OLAP vs. OLTP).
- And ***how we format the data***
  - Human-readable format may not bring us the performance we are after.
  - The reason behind ETL. What does ETL stand for?
    - More recently ELT is more popular – we will cover that more later in class (link).

- Not all tools offer the same power over processing the data – as we will see later in class.

Cache-Friendly Code

Programmer can optimise for cache performance

- How data structures are organised (alignment and layout)
- How data are accessed:
  - Nested loop structure
  - Blocking is a general technique

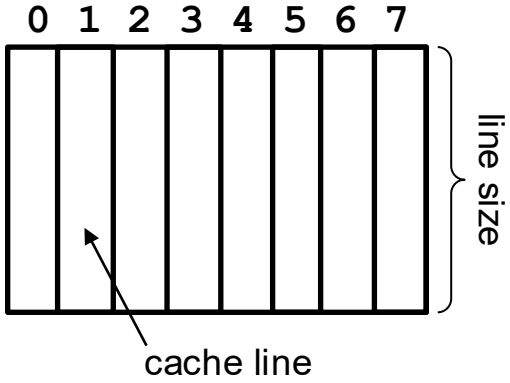
All systems favour “cache-friendly code”

- Can get most of the advantage with generic code:
  - Keep working set reasonably small (temporal locality)
  - Use small strides (spatial locality)
  - Focus on inner loop cycle

Quick re-cap of CPU cache internals

To guarantee speed, caches move data in fixed-size units.

- Organize cache in ***cache lines***.
- Only load / evict ***full cache lines***.
- Typical ***cache line size*** is 64 bytes.



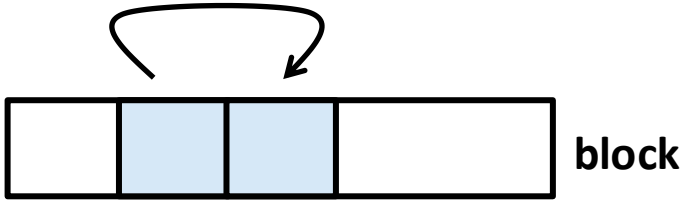
- We should try to make use of everything fetched / stored within a cache line.
- CPUs often leverage a pre-fetcher that monitors memory access patterns (e.g., a sequential or strided) and speculatively issues memory requests for future cache lines before the CPU loads from them.

Principle of Locality

Caches take advantage of the ***principle of locality***

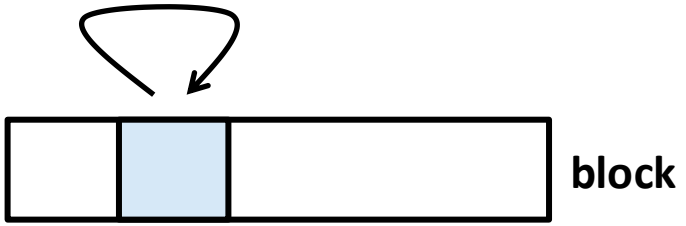
Spatial locality:

- Related data is often spatially close
- Stride-1 arrays, sequential instructions, etc.



Temporal locality:

- Code may call a function repeatedly, even if it is not spatially close
- Programs tend to reuse data frequently.
  - e.g., the hot set of data often fits into caches



Example locality: Data? Instructions?

```
sum = 0;
for (i = 0; i < n; i++)
{
    sum += a[i];
}
return sum;
```

Temporal locality:

- Data: sum referenced in each iteration cycle through loop repeteadly
- Instructions:

Spatial locality:

- Data:
- Instructions:

Locality example

How we access data stored in memory can have significant impact on performance.

```
int sum_array_col(int a[M][N])
{
  int i, j, sum = 0;
  for (j = 0; j < N; j++) {
    for (i = 0; i < M; i++) {
      sum += a[i][j];
    }
  }
  return sum;
}
```

```
int sum_array_rows(int a[M][N])
{
  int i, j, sum = 0;
  for (i = 0; i < M; i++) {
    for (j = 0; j < N; j++) {
      sum += a[i][j];
    }
  }
  return sum;
}
```

Locality example

Executing the program for a 20’000 x 20’000 matrix gives an order of magnitude difference:

- ***16.98 sec*** for `sum_array_col` vs ***1.71 sec*** for `sum_array_row`

```
# Samples: 89K of event 'cpu-clock'
# Event count (approx.): 22473000000
#
# Overhead       Samples  Command  Shared Object      Symbol
# ........  ............  .......  .................  ...................................
#
    75.54%         67903  a.out    a.out              [.] sum_array_col
     7.61%          6844  a.out    a.out              [.] sum_array_row
     5.59%          5021  a.out    a.out              [.] main
     3.65%          3277  a.out    libc-2.27.so       [.] __random
     3.35%          3012  a.out    libc-2.27.so       [.] __random_r
     0.88%           795  a.out    libc-2.27.so       [.] rand
     0.85%           763  a.out    [kernel.kallsyms]  [k] __do_page_fault
     0.44%           393  a.out    a.out              [.] rand@plt
     0.43%           390  a.out    [kernel.kallsyms]  [k] clear_page_erms
     0.28%           256  a.out    [kernel.kallsyms]  [k] _raw_spin_unlock_irqrestore
```

```
# Samples: 89K of event 'cache-misses'
# Event count (approx.): 1177848296
#
# Overhead       Samples  Command  Shared Object      Symbol
# ........  ............  .......  .................  ...................................
#
    87.45%         67527  a.out    a.out              [.] sum_array_col
     6.93%          6840  a.out    a.out              [.] sum_array_row
     2.30%          6387  a.out    [kernel.kallsyms]  [k] clear_page_erms
     0.95%          2636  a.out    [kernel.kallsyms]  [k] _raw_spin_lock
     0.28%           754  a.out    [kernel.kallsyms]  [k] mem_cgroup_try_charge
     0.27%           760  a.out    [kernel.kallsyms]  [k] mem_cgroup_throttle_swaprate
     0.25%           700  a.out    [kernel.kallsyms]  [k] get_page_from_freelist
     0.16%           467  a.out    [kernel.kallsyms]  [k] try_charge
     0.13%           377  a.out    [kernel.kallsyms]  [k] prep_new_page
```

Quick check with `perf` measuring the `cpu cycles` and `cache misses` confirms the importance of writing ***cache-friendly*** code.

Locality example #1

```
int sum_array_cols(int a[M][N])
{
  int i, j, sum = 0;
  for (j = 0; j < N; j++) {
    for (i = 0; i < M; i++) {
      sum += a[i][j];
    }
  }
  return sum;
}
```

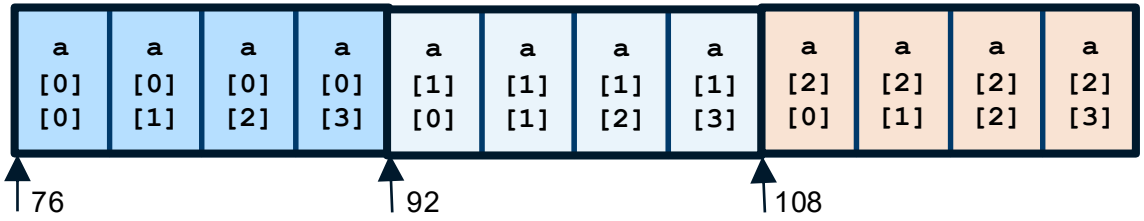
M = 3, N = 4

a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]

Access Pattern:
stride = ?

1)	a[0][0]
2)	a[1][0]
3)	a[2][0]
4)	a[0][1]
5)	a[1][1]
6)	a[2][1]
7)	a[0][2]
8)	a[1][2]
9)	a[2][2]
10)	a[0][3]
11)	a[1][3]
12)	a[2][3]

Layout in Memory



Note: 76 is just one possible starting address of array a

Locality example #2

```
int sum_array_rows(int a[M][N])
{
  int i, j, sum = 0;
  for (i = 0; i < M; i++) {
    for (j = 0; j < N; j++) {
      sum += a[i][j];
    }
  }
  return sum;
}
```

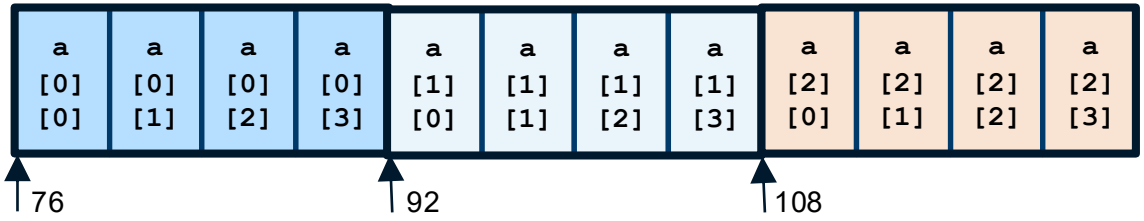
M = 3, N = 4

a[0][0]	a[0][1]	a[0][2]	a[0][3]
a[1][0]	a[1][1]	a[1][2]	a[1][3]
a[2][0]	a[2][1]	a[2][2]	a[2][3]

Access Pattern:
stride = ?

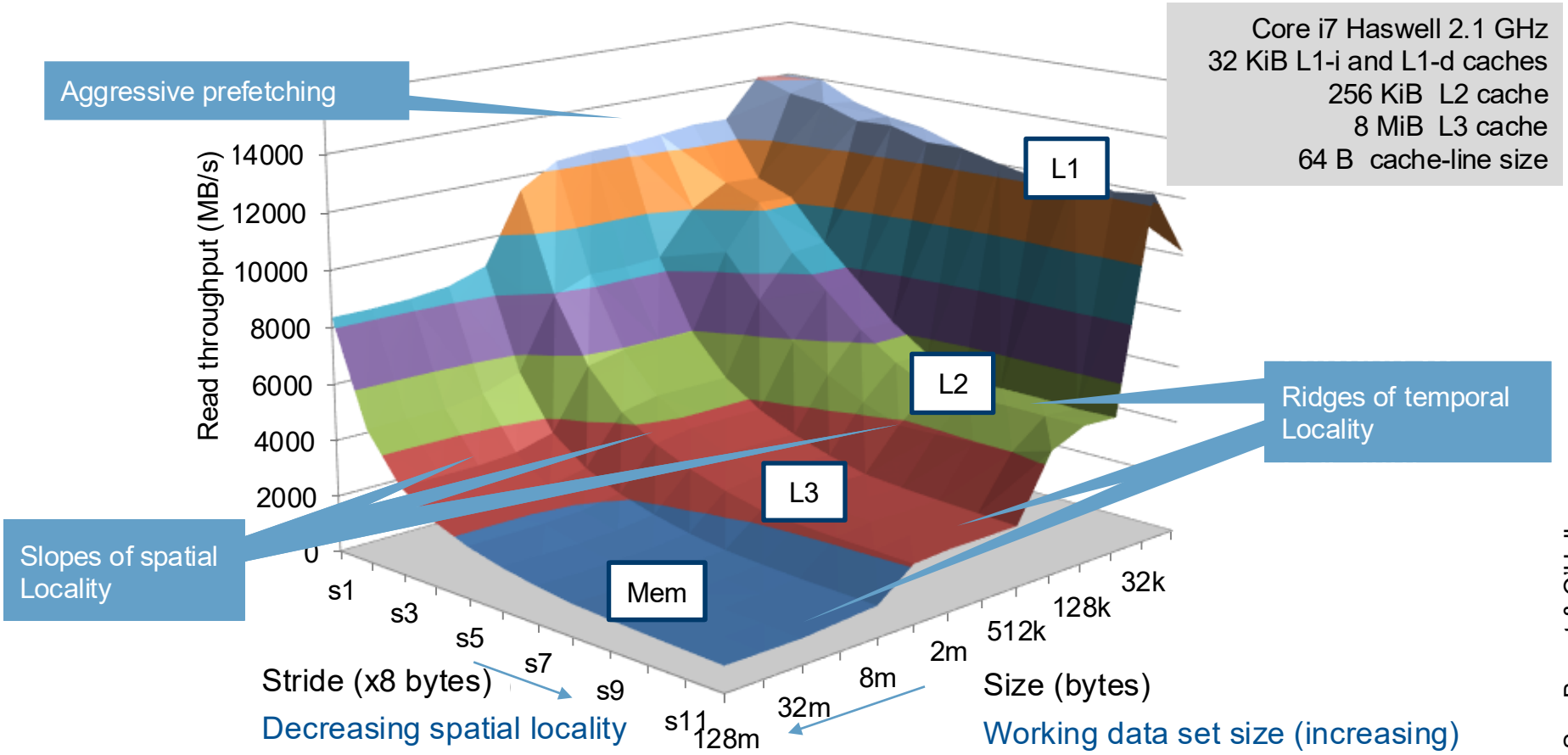
1) a[0][0]
2) a[0][1]
3) a[0][2]
4) a[0][3]
5) a[1][0]
6) a[1][1]
7) a[1][2]
8) a[1][3]
9) a[2][0]
10) a[2][1]
11) a[2][2]
12) a[2][3]

Layout in Memory



Note: 76 is just one possible starting address of array a

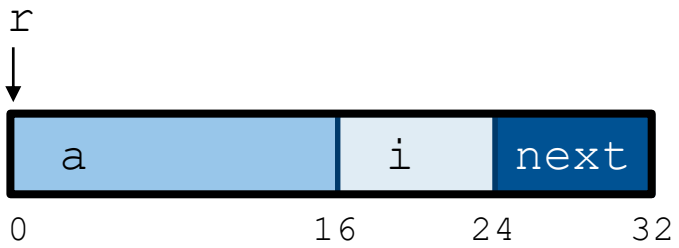
The Memory Mountain



Src: Bryant & O'Hallaron

Structure Representation

```
typedef struct rec {
  int a[4];
  long i;
  struct rec *next;
} *r;
```



Structure is represented as block of memory:

- Big enough to hold all of the fields

Fields ordered according to declaration order

- Even if another ordering would be more compact

Compiler determines overall size + positions of fields

- Machine-level programs have no understanding of the structures in the source code

Memory Alignment in x86-64

For good memory system performance, Intel recommends data to be aligned

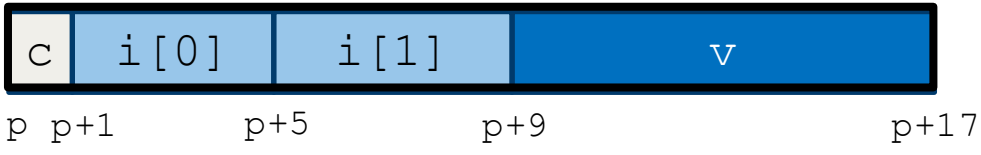
- Memory is accessed in word-chunks, so it is inefficient to load/store values that span word boundaries and especially cache-line boundaries

- However, the x86-64 hardware will work correctly regardless of alignment of data

Aligned means that any primitive object of *K* bytes must have an address that is multiple of *K*

K	Type	Addresses
1	`char`	No restrictions
2	`short`	Lowest bit must be zero: $...0_2$
4	`int, float`	Lowest 2 bits zero: $...00_2$
8	`long, double`	Lowest 3 bits zero: $...000_2$
16	`long double`	Lowest 4 bits zero: $...0000_2$

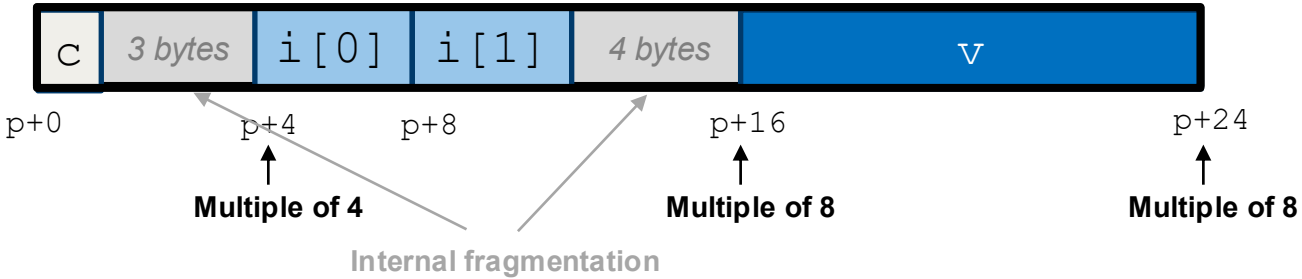
Structures and Alignment




```
struct S1 {
  char c;
  int i[2];
  double v;
} *p;
```

Aligned Data:

- Primitive data type requires *K* bytes
- Address must be multiple of *K*



Even though it is not packed, this padded data structure will result in better performance.

Alignment of Structs

Compiler will do the following:

- Maintains declared ***ordering*** of fields in struct
- Each ***field*** must be aligned ***within*** the struct *(may insert padding)*
  - `offsetof` can be used to get actual field offset
- Overall struct must be ***aligned*** according to largest field
- Total struct ***size*** must be multiple of its alignment *(may insert padding)*
  - `sizeof` should be used to get true size of `structs`
- For strings and other variable-length data (depending on their class definitions)
  - split the string into length and data: fixed size header and variable size tail.
  - header contains pointers to tail.
  - place variable data at the end of the `struct` (consider as alignment 1)
  - cf. Database Systems on Modern CPU Architectures (Access Paths)

How you can save space

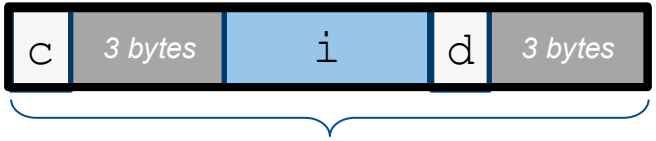
The compiler must respect the order elements are declared in

- Sometimes the programmer can save space by declaring large data types first

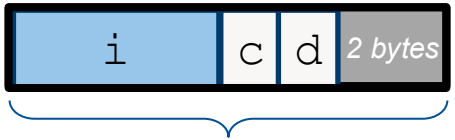
```
struct S4 {
  char c;
  int i;
  char d;
} *p;
```



```
struct S5 {
  int i;
  char c;
  char d;
} *p;
```



12 bytes



8 bytes

Data alignment

Task: test effect of padding and alignment when inserting tuples in an array (single socket, 4 hw-threads)

```
struct S1 {
  int primary_key;
  long timestamp;
  char color[2];
  int zipcode;
} *p;
```



Alignment	Throughput
No Alignment	0.523 MB/s
Padding	11.7 MB/s
Reordering + Padding	814.8 MB/s

→ src: CMU-DB Alignment Experiment by Tianyu Li

Array of Structs vs. Struct of Arrays

- Another way to look at ***row-store*** vs. ***column store***

- **Array of Structures (AoS)** → stores a complete record in contiguous memory
- **Structure of Arrays (SoA)** → separates the fields into distinct arrays, where each array

- Q: which one would be better for *updates?*
- Q: which one is more suitable for *reads?*

- Simple experiment, 100 million records of Struct `persons (id, name, degree, salary)`.
  - *Updating* 10m records, change their 'degree' and the affected 'salary'
    - AoS – 126.58ms
    - SoA – 195.66ms
  - *Reading* the persons' salaries to sum the total salary
    - AoS – 137.22ms
    - SoA – 45.67ms

https://cedardb.com/blog/optimizing_data_layouts/

Quick recap of virtual memory and address translation

Virtual memory

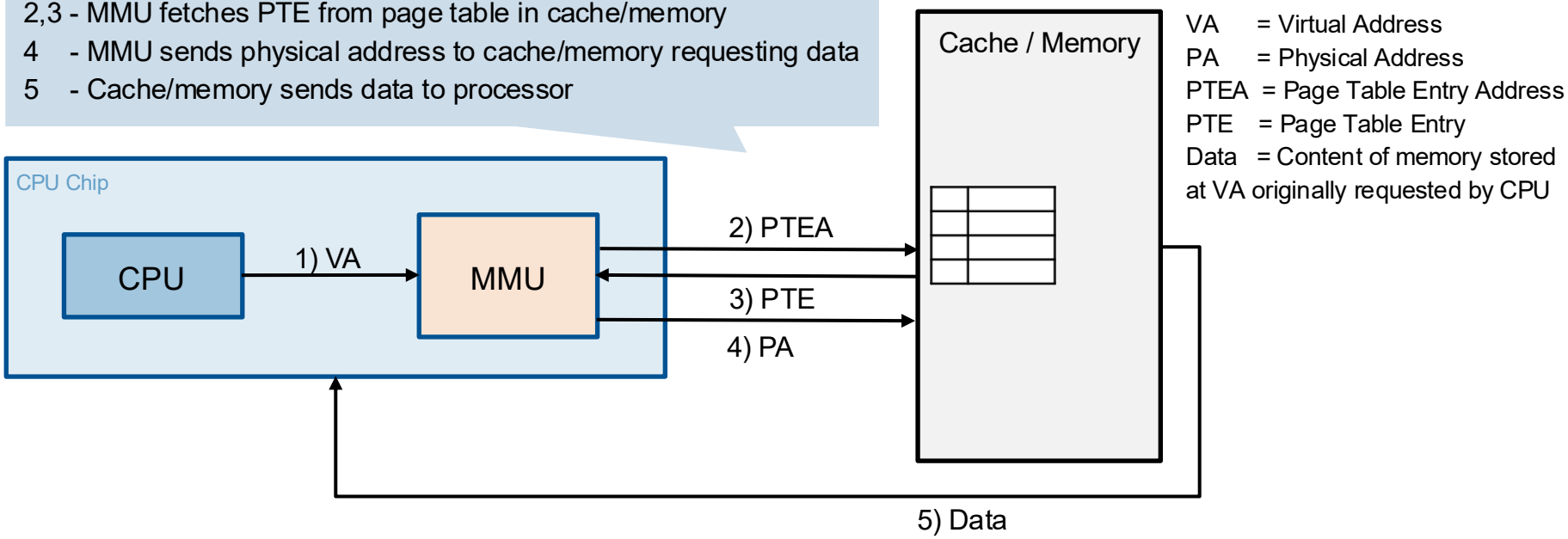
- Each user-space ***process*** has its own ***virtual memory*** – an ***illusion*** that it runs alone on the computer and has access to the computer's full memory.

- **Q: Why?**
  - Allows for inter user-space process *isolation* and *protection*.
    - Each process runs in its own virtual address space; others cannot read or overwrite.
  - It *simplifies memory management* with a contiguous address space
    - physical memory may be scattered across many addresses.
  - *Efficient use of the physical memory (DRAM)*
    - Virtual memory allows the Operating System to page data in and out of physical DRAM.
  - *Memory sharing*
    - Different processes can share parts of memory safely by mapping the same physical pages into multiple virtual address spaces (`mmap`) – will use it in the upcoming lecture.

Memory translation

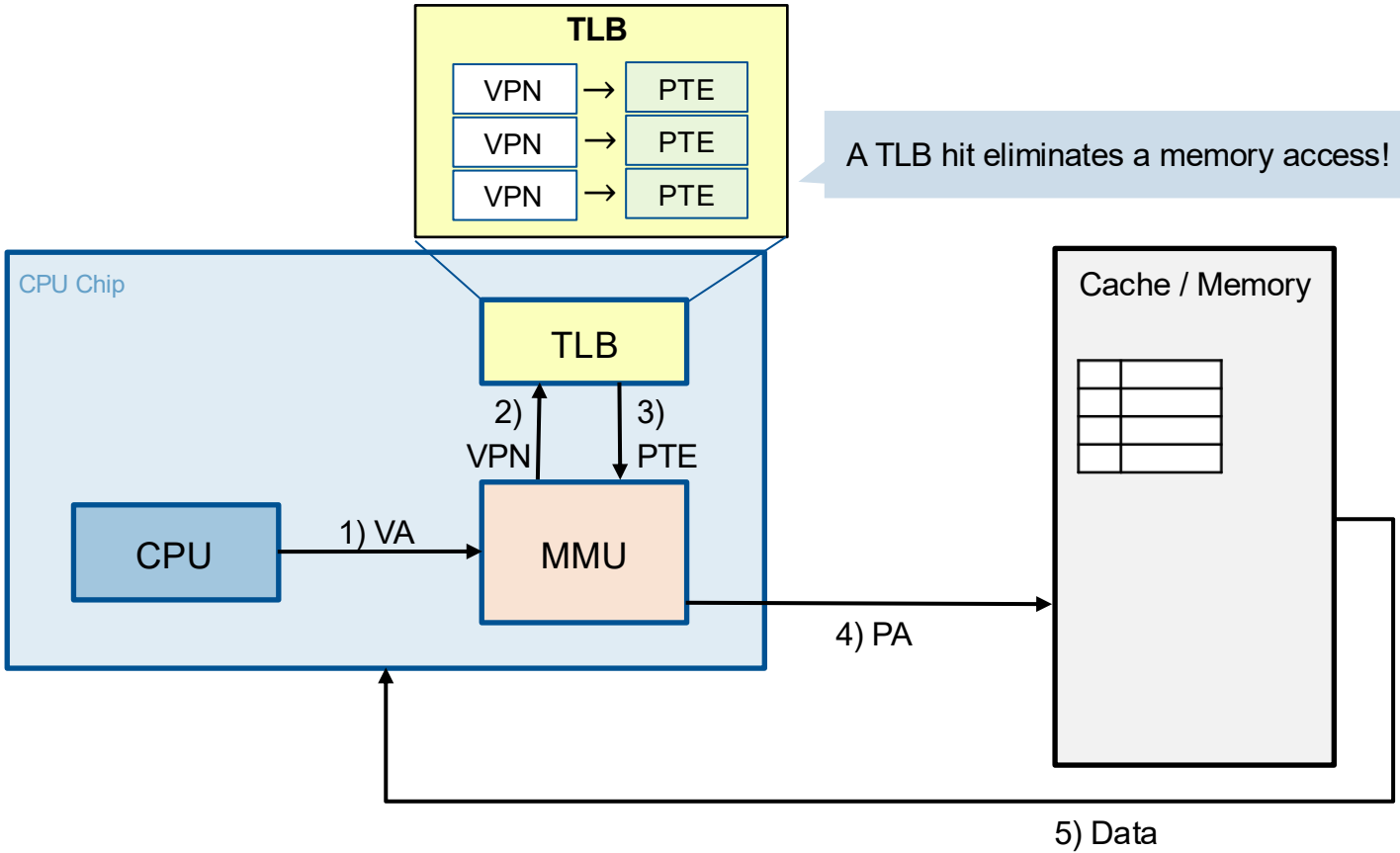
- Request is virtual address (VA), want physical address (PA)
- Use look-up table that we call ***page table (PT)***

1 - Processor sends virtual address to MMU
2,3 - MMU fetches PTE from page table in cache/memory
4 - MMU sends physical address to cache/memory requesting data
5 - Cache/memory sends data to processor

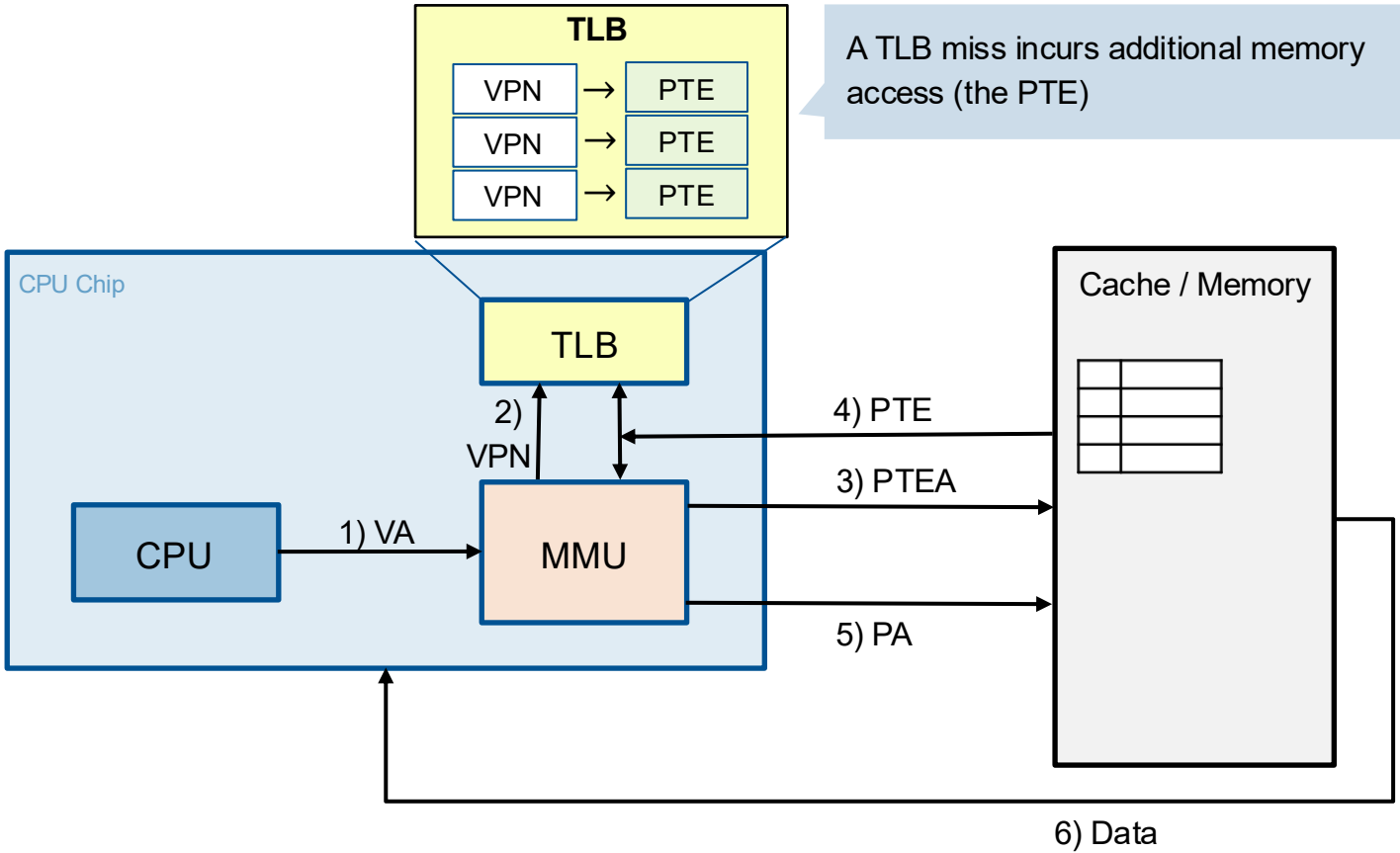


VA = Virtual Address
PA = Physical Address
PTEA = Page Table Entry Address
PTE = Page Table Entry
Data = Content of memory stored at VA originally requested by CPU

Translation Lookaside Buffer (TLB)



TLB Miss


TLB
VPN → PTE
VPN → PTE
VPN → PTE
A TLB miss incurs additional memory access (the PTE)
CPU Chip
TLB
2) VPN
CPU
1) VA
MMU
4) PTE
3) PTEA
5) PA
Cache / Memory
6) Data

Key take-away

- **Modern machines are very powerful**, but to fully exploit their potential requires:
  - Understanding their characteristics
  - Careful placement of data (or choosing the right format / data structure)
  - Traversing the data in a cache-friendly way (or adjusting the algorithm accordingly)
  - Being mindful of the key latency numbers (bandwidth/capacity/cost)

- **Only when we know how to efficiently use a single machine (thread) should we consider scaling (next lecture).**

References

- Some material taken from:
  - Lecture: *Data Processing on Modern Hardware* – offered by our Chair in Summer Semester
  - Lecture: *Introduction to Computer Architecture* – Prof. Jana Giceva (while at *Imperial College London*)
  - Lecture: *Data Processing on Modern Hardware* – Prof. Jens Teubner (TU Dortmund)
  - Lecture: *Advanced Databases* – Prof. Andy Pavlo (CMU)

- Other materials:
  - Paper: *Scalability! But at what COST?* (HotOS'15)
  - Book: *Computer Systems: A Programmer's Perspective* by Bryant and O'Hallaron
  - Book: *What every programmer should know about memory* by Ulrich Drepper